Lecture 13
Functional synthesis with Sketch

Armando Solar-Lezama

Example

You want to partition N elements over P procs

- How many elements should a processor get?



N = 18
P = 5

Obvious answer is N/P

Obvious answer is wrong!

Synthesizing a partition function

What do we know?

- The interface to the function we want
- Not all processors will get the same # of elements
- The kind of expressions we expect

```
void partition(int p, int P, int N, ref int ibeg, ref int iend){

    ` -                N       P        N%P
                         *       +
                   p             N/P

}
```

Synthesizing a partition function

How does the system know what a partition is?

```
harness void testPartition(int p, int N, int P){
    int ibeg, iend;

}
```

Partitions should be balanced

Adjacent partitions should match

First and last partition should go all the way to the ends

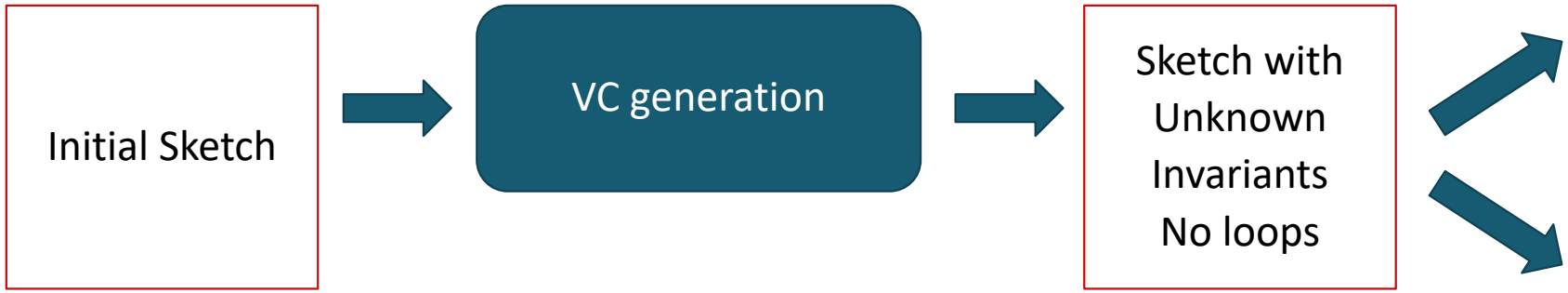
Challenges

Loops and Recursion

CEGIS failures

Limitations of Theory Reasoning

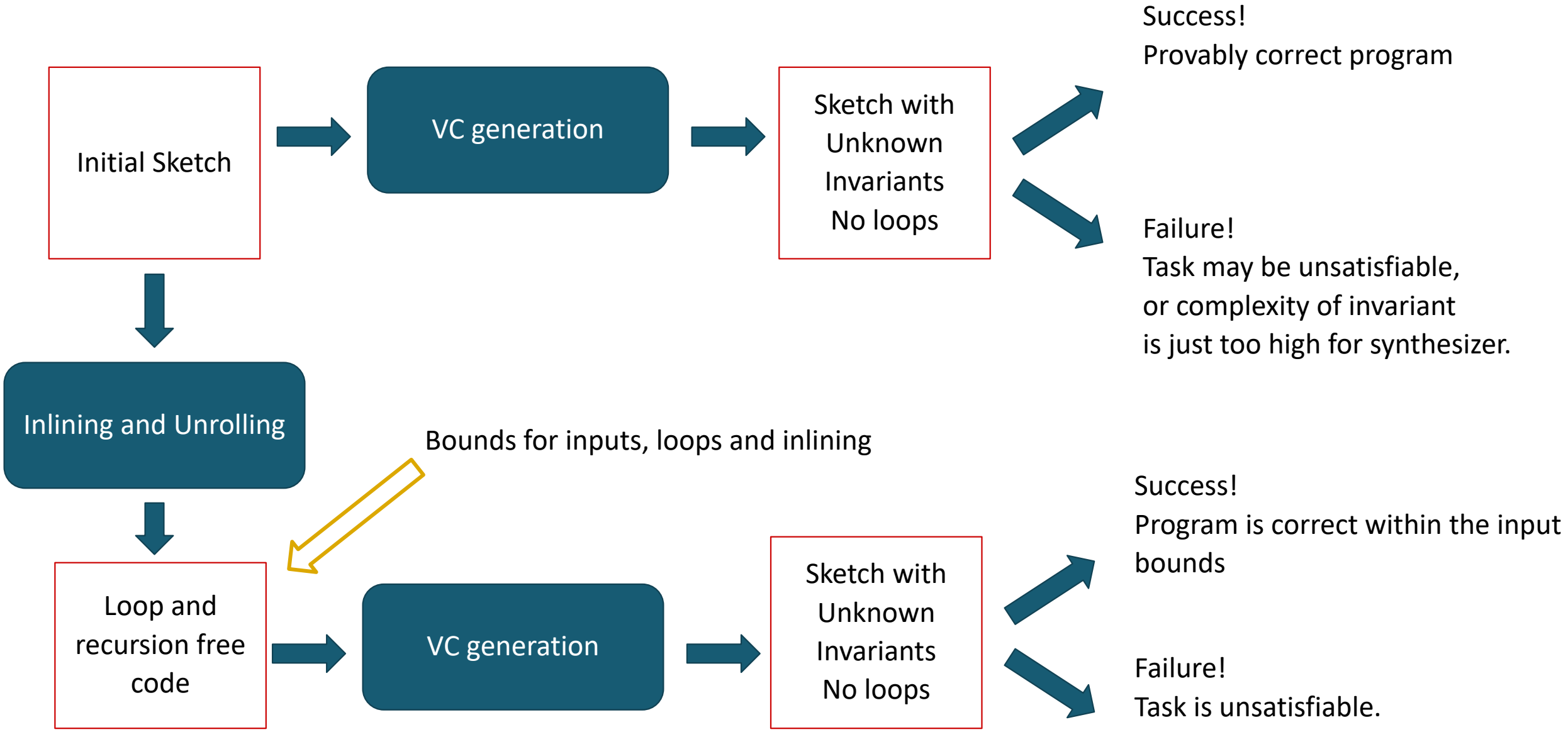
Coping with loops



Success!
Provably correct program

Failure!
Task may be unsatisfiable,
or complexity of invariant
is just too high for synthesizer.

Coping with loops



Strategy

Unroll all loops

Inline all functions

Now you have a loop free program

- The method of generating Verification conditions is now fully algorithmic

Loop Unrolling

```
while(b){
    C
}
```

≡

```
if(b){
  C
  while(b){
    C
  }
}
```

Loop Unrolling

```
while(b){
    C
}
```

≡

```
if(b){
  C
  while(b){
    C
  }
}
```

```
while(b){
    C
}
```

→

```
if(b){
  C
  while(b){
    C
  }
}
```

Loop Unrolling

```
while(b){
    C
}
```

≡

```
if(b){
  C
  while(b){
     C
  }
}
```

```
while(b){
    C
}
```

→

```
if(b){
  C
  while(b){
     C
  }
}
```

→

```
if(b){
  C
  if(b){
     while(b){
        C
     }
  }
}
```

Loop Unrolling

```
while(b){
    C
}
```

≡

```
if(b){
  C
  while(b){
    C
  }
}
```

```
while(b){
    C
}
```

→

```
if(b){
  C
  while(b){
    C
  }
}
```

→

```
if(b){
  C
  if(b){
    while(b){
      C
    }
  }
}
```

• • •

→

```
...
  if(b){
    C
    if(b){
      assert false;
    }
  }
}
```

Loop Unrolling

```
while(b){
    C
}
```

≡

```
if(b){
  C
  while(b){
    C
  }
}
```

```
while(b){
    C
}
```

→

```
if(b){
  C
  while(b){
    C
  }
}
```

→

```
if(b){
  C
  if(b){
    while(b){
      C
    }
  }
}
```

• • • →

```
...
  if(b){
    C
    if(b){
      assert false;
    }
  }
}
```

Why not assume?



Pros vs. Cons

Pros of unrolling approach

- Can find solutions even for cases where the invariants are too complex to synthesize
- Eliminate risk of non-terminating solution without the need for termination proof

Cons

- Formulas can grow really big
- A lot of “magic numbers” needed to coordinate input bounds, with unrolling bounds of many different loops.
- Solution may be incorrect

CEGIS Failure

Comp0(in, out)

Comp1(in, out)

Comp2(in, out)

I want to synthesize an arbitrary permutation of the three components.

```
generator void(State in, ref State out){
    State locIn = in;
    State locOut;
    repeat(3){
        {| Comp0(locIn, locOut)
         |  Comp1(locIn, locOut)
         |  Comp2(locIn, locOut)
         |}
        locIn = locOut;
    }
    out = locOut;
}
```

Simple sketch but fails to enforce that each component is used exactly once.

CEGIS Failure

Comp0(in, out)

Comp1(in, out)

Comp2(in, out)

I want to synthesize an arbitrary permutation of the three components.

Sketch enforces the constraint of using each component once, but is more complex and still has the problem that the resulting formula ends up with 3 copies of each component.

```
generator void(State in, ref State out){
    State locIn = in;
    State locOut;
    bit[3] used;
    repeat(3){
        if(??){ assert !used[0];
                Comp0(locIn, locOut);
                used[0] = 1;
        } else
        if(??){ assert !used[1];
                Comp1(locIn, locOut);
                used[1] = 1;
        } else
        if(??){ assert !used[2];
                Comp2(locIn, locOut);
                used[2] = 1;
        }
     }
    out = locOut;   assert used == {1,1,1};
```

CEGIS Failure

Comp0(in, out)

Comp1(in, out)

Comp2(in, out)

I want to synthesize an arbitrary permutation of the three components.

tin is an additional input that we get from calling an uninterpreted function. This version of the problem only requires one copy of each component, but causes CEGIS to work very inefficiently.

```
generator void(State in, ref State out){
    State[3] tin = NonDeterministic();

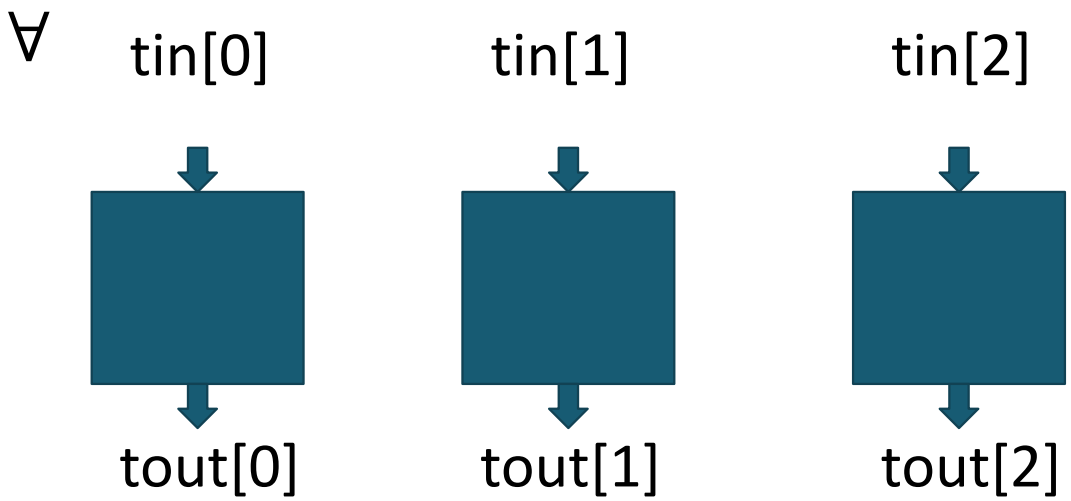
    int fst = ??; // Component in the first pos.
    int snd = ??; //Component in the second pos.
    int thrd = ??; //Component in the third pos.
    assert fst < 3;
    assert snd < 3 && snd != fst;
    assert thrd < 3 && thrd != snd && thrd != fst;

     Comp1(tin[0], tout[0]);
     Comp2(tin[1], tout[1]);
     Comp3(tin[2], tout[2]);

    assume tin[fst] == in;
    assume tin[snd] == tout[fst];
    assume tin[thrd] == tout[snd];

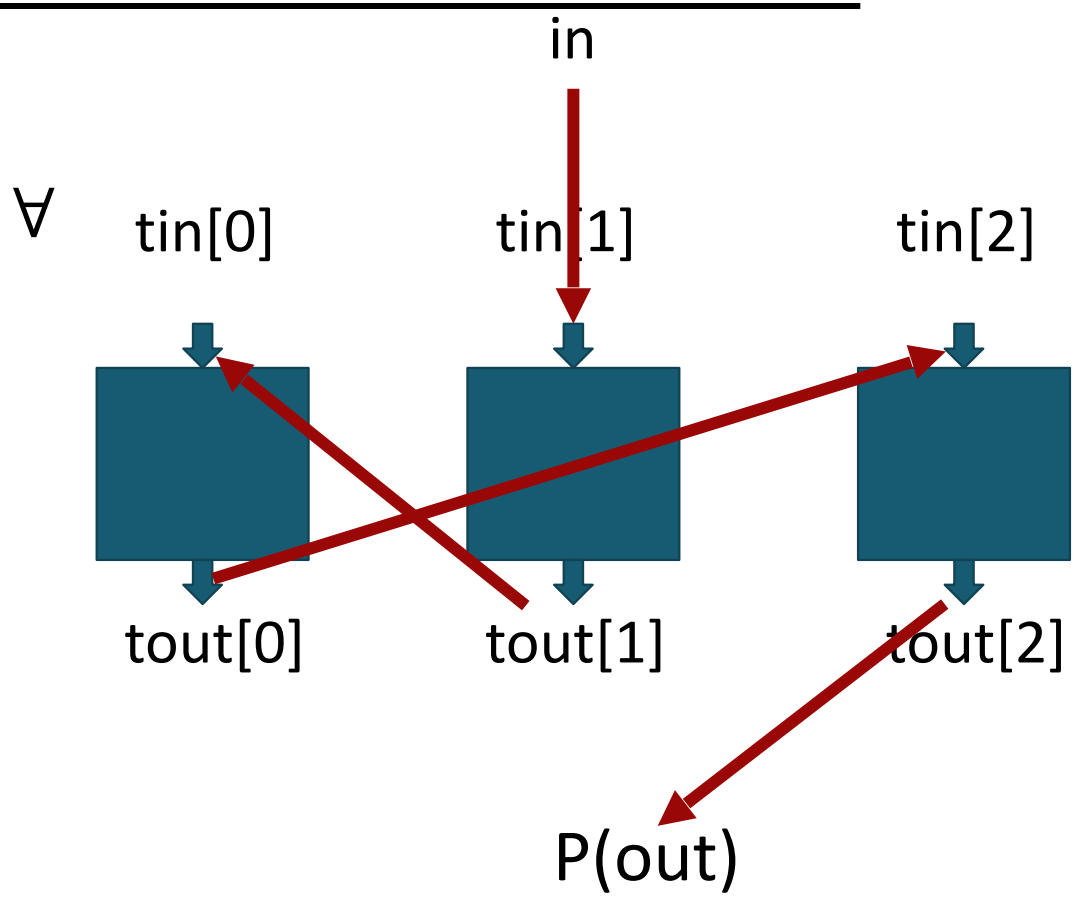
    out = tout[thrd];
}
```

CEGIS Failure



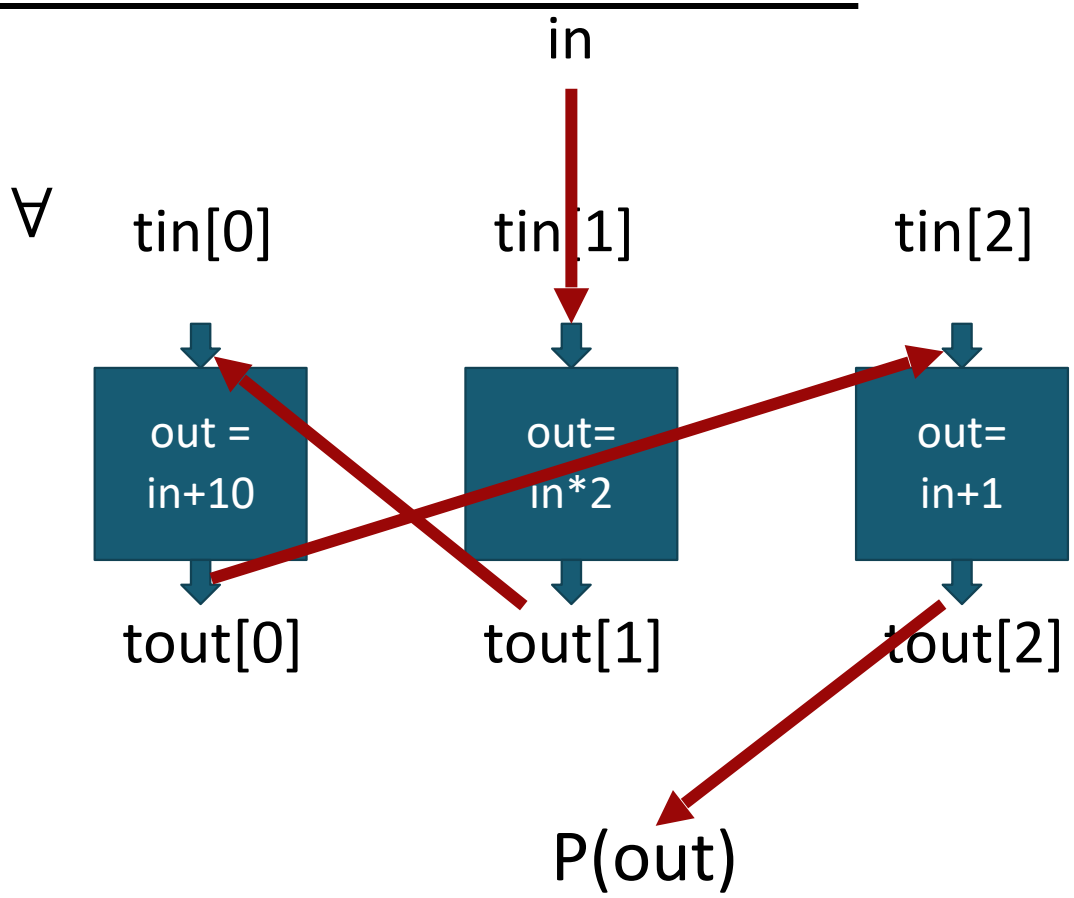

```
if(in==tin[fst] &&
   tin[snd] == tout[fst] &&
   tin[thrd] ==tout[snd]){
      assert P(tout[thrd]);
}
```

CEGIS Failure




```
if(in==tin[1] &&
   tin[0] == tout[1] &&
   tin[2] ==tout[0]){
      assert P(tout[2]);
}
```

CEGIS Failure

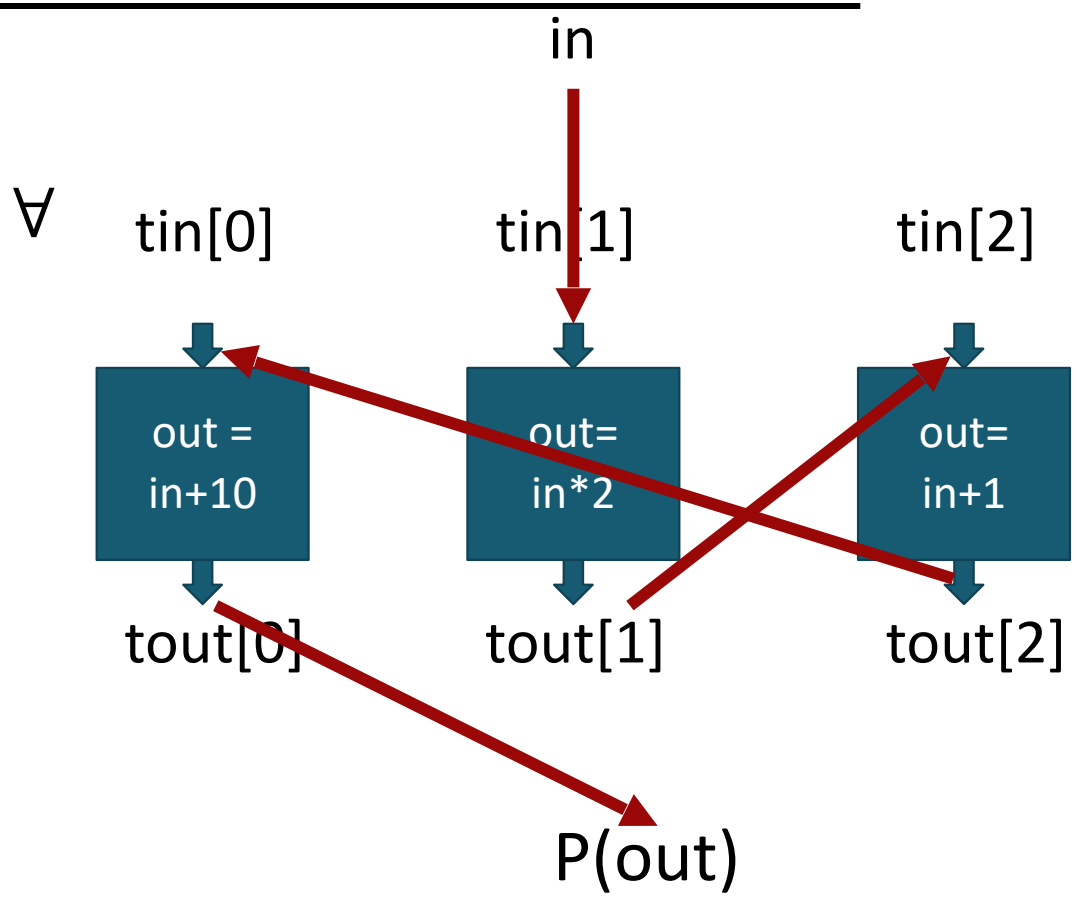


Counterexample 1:
in = 5
tin[1] = 5
tin[0] = 10
tin[2] = 20

```
if(in==tin[1] &&
   tin[0] == tout[1] &&
   tin[2] ==tout[0]){
      assert P(tout[2]);
}
```

Assert out = (in+10)*2 + 1

CEGIS Failure



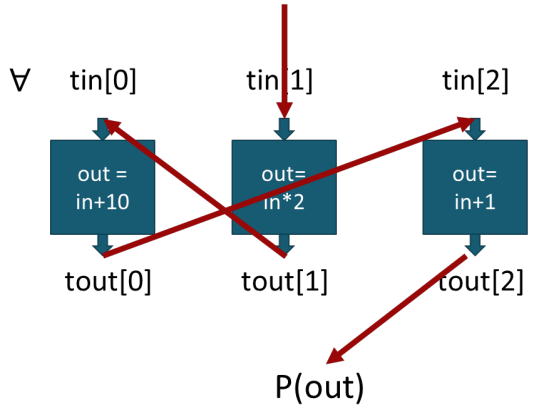

```
if(in==tin[1] &&
   tin[2] == tout[1] &&
   tin[0] ==tout[2]){
      assert P(tout[0]);
}
```

Assert out = (in+10)*2 + 1

Counterexample 1:
- in = 5
- tin[1] = 5
- tin[0] = 10
- tin[2] = 20

Counterexample 2:
- in = 5
- tin[1] = 5
- tin[0] = 11
- tin[2] = 10

CEGIS Failure



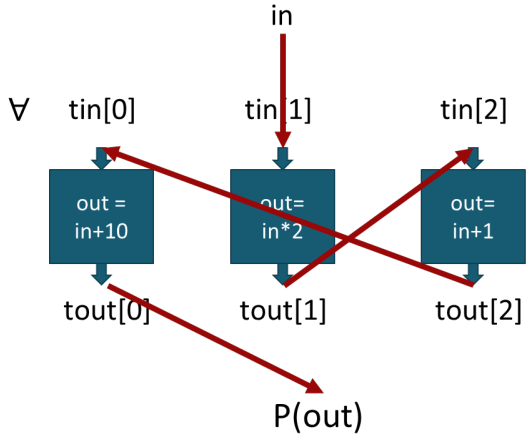
Counterexample 1:

in = 5

tin[1] = 5

tin[0] = 10

tin[2] = 20



Counterexample 2:

in = 5

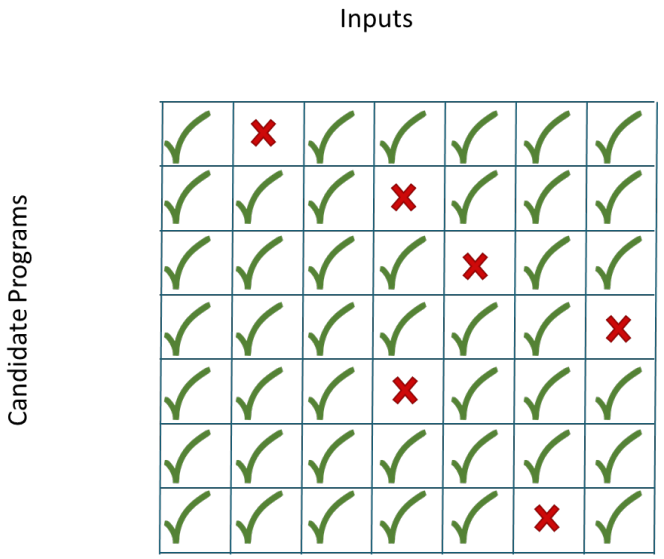
tin[1] = 5

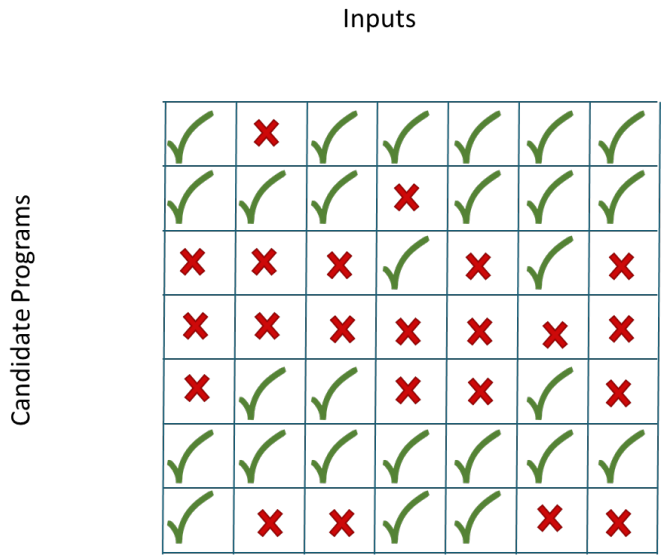
tin[0] = 11

tin[2] = 10

Counterexample rules out candidate that produced it, but it cannot even rule out an equivalent candidate

CEGIS Failure





General problem

$$\exists \phi\,.\ \forall in, temp\,.\ P\big(\phi,\ in,\ temp\big) \Rightarrow Q(\phi,\ in,\ temp)$$

When $P(\phi,\ in,\ temp)$ is hard to satisfy and is under the control of ϕ it becomes easy for the synthesizer to satisfy the constraint by making P false.

- Ex. if P is $in == temp + \phi(1)$ you will have this problem as well changing $\phi(1)$ invalidates the current counterexample.

Idea

$$\exists \phi\,.\ \forall in, temp\,.\ P\big(\phi,\ in,\ temp\big) \Rightarrow Q(\phi,\ in,\ temp)$$

Consider instead the formula:

$$\exists \phi\,.\ \forall in\ \exists temp\,.\ P\big(\phi,\ in,\ temp\big) \wedge Q(\phi,\ in,\ temp)$$

The two formulas are not equivalent, but can be equivalent in the case where $\forall \phi,\ in$ there is one and only one tmp that satisfies $P(\phi,\ in,\ temp)$

Adding quantifiers makes it easier

For the synthesis problem, the quantifier goes away

- Given inputs $in_1,\ in_2$ you get the formula:

$$\exists \phi\, \exists tmp_1,\ tmp_2\,.\ P\big(\phi,\ in_1,\ temp_1\big) \wedge Q\big(\phi,\ in_1,\ temp_1\big) \\ \wedge P\big(\phi,\ in_2,\ temp_2\big) \wedge Q(\phi,\ in_2,\ temp_2)$$